

Büyük Ölçekli Sırt Çantası (Knapsack) Problemlerinde Genetik Algoritma ile Dinamik Programlamanın Zaman Karmaşıklığı Karşılaştırması

Melisa ÇAKICI, Zeren KARAKUŞ, Umut ŞAHİN

Yazılım Mühendisliği Bölümü

Hasan Ferdi Turgutlu Teknoloji Fakültesi

Manisa Celal Bayar Üniversitesi

Email: {232803033,232803044,232805002}@ogr.cbu.edu.tr

Özet—Bu çalışmada, NP-Hard problem sınıfının en temel örneklerinden biri olan Sırt Çantası Problemi (Knapsack Problem) üzerinde Dinamik Programlama (DP) ve Genetik Algoritma (GA) yöntemlerinin performansları analiz edilmiştir. Çalışmanın temel amacı, veri boyutu arttıkça kesin çözüm yöntemlerinin karşılaştığı donanım kısıtlarını ve sezgisel yöntemlerin sağladığı ölçeklenebilirliği incelemektir. Deneyler $N=100$, $N=1000$ ve $N=10.000$ elemanlı veri setleri üzerinde gerçekleştirilmiştir. Elde edilen sonuçlar, Dinamik Programlama yönteminin büyük ölçekli verilerde zaman ve bellek maliyeti açısından verimsiz hale geldiğini, Genetik Algoritmanın ise saniyeler içerisinde kabul edilebilir doğrulukta çözümler üretebildiğini göstermektedir.

Index Terms—Knapsack Problemi, Dinamik Programlama, Genetik Algoritma, NP-Hard, Algoritma Analizi

I. GİRİŞ (INTRODUCTION)

Sırt Çantası Problemi (Knapsack Problem), belirli bir kapasite sınırı altında maksimum değeri elde etmeyi amaçlayan kombinatoriyal optimizasyon problemlerinden biridir [1]. Problem, lojistik, kaynak planlama, yatırım optimizasyonu ve veri yönetimi gibi birçok alanda kullanılmaktadır.

Knapsack problemi matematiksel olarak NP-Hard problem sınıfında yer almaktadır [2]. Bu nedenle veri boyutu arttıkça çözüm uzayı üstel olarak büyümekte ve klasik algoritmaların çalışma süreleri ciddi şekilde artmaktadır.

Dinamik Programlama gibi kesin çözüm yöntemleri küçük ve orta ölçekli problemlerde optimal çözüm üretirken, büyük ölçekli veri setlerinde yüksek zaman ve bellek maliyetleri oluşturmaktadır [3]. Buna karşılık Genetik Algoritma gibi sezgisel yöntemler optimum çözümü garanti etmese de çok daha kısa sürelerde yakın sonuçlar üretebilmektedir [4].

Bu çalışmada Dinamik Programlama ve Genetik Algoritma yöntemlerinin büyük ölçekli veri setleri üzerindeki performansları karşılaştırılmıştır.

II. LİTERATÜR TARAMASI (LITERATURE REVIEW)

Literatürde Knapsack probleminin çözümü için kesin (Exact) ve sezgisel (Heuristic) olmak üzere iki temel yaklaşım bulunmaktadır [3].

Dinamik Programlama yöntemi, problemin optimal alt yapı özelliğinden yararlanarak kesin çözüm üretmektedir. Ancak yöntemin çalışma karmaşıklığı:

$$O(N \times W)$$

şeklinde dir. Burada N eşya sayısını, W ise çanta kapasitesini temsil etmektedir. Bu nedenle kapasite değeri büyüdükçe yöntem yüksek bellek tüketimine neden olmaktadır.

Genetik Algoritmalar ise doğal seçim mekanizmasını temel alan sezgisel optimizasyon yöntemleridir [4]. Bu yöntemler optimum çözümü garanti etmez ancak büyük veri setlerinde oldukça hızlı sonuç üretebilirler.

Son yıllarda yapılan çalışmalar, özellikle büyük ölçekli NP-Hard problemlerde sezgisel yöntemlerin daha ölçeklenebilir çözümler sunduğunu göstermektedir [6].

III. YÖNTEM (METHOD)

Bu çalışmada aynı veri setleri üzerinde Dinamik Programlama ve Genetik Algoritma yöntemleri uygulanarak performans karşılaştırması gerçekleştirilmiştir.

A. Knapsack Probleminin Matematiksel Modeli

Knapsack problemi aşağıdaki şekilde modellenmektedir:

$$\max \sum_{i=1}^n v_i x_i$$

$$\sum_{i=1}^n w_i x_i \leq W$$

$$x_i \in \{0, 1\}$$

Burada:

- v_i : eşyanın değeri
- w_i : eşyanın ağırlığı
- x_i : eşyanın seçilip seçilmediği
- W : çanta kapasitesi

B. Dinamik Programlama (DP)

Dinamik Programlama yöntemi problemi daha küçük alt problemlere ayırarak çözmektedir [5].

DP yaklaşımında her kapasite değeri için maksimum değer bir tabloda saklanmaktadır. Algoritma aşağıdaki geçiş formülünü kullanmaktadır:

$$DP[i][w] = \max(DP[i-1][w], v_i + DP[i-1][w - w_i])$$

Bu yöntem teorik olarak her zaman optimal sonucu vermektedir. Ancak veri boyutu arttıkça tablo boyutu da ciddi şekilde büyümektedir.

C. Genetik Algoritma (GA)

Genetik Algoritma doğal evrim sürecini taklit eden sezgisel bir optimizasyon yöntemidir [4].

Bu çalışmada kullanılan GA yapısı aşağıdaki adımlardan oluşmaktadır:

- **Kodlama:** Bireyler 0 ve 1 dizileri şeklinde temsil edilmiştir.
- **Başlangıç Popülasyonu:** Rastgele bireylerden oluşan başlangıç popülasyonu oluşturulmuştur.
- **Seçilim:** Turnuva seçilim yöntemi kullanılmıştır.
- **Çaprazlama:** Tek noktalı çaprazlama yöntemi uygulanmıştır.
- **Mutasyon:** Yerel optimuma takılmayı önlemek amacıyla %5 mutasyon oranı kullanılmıştır.

IV. DENEYSEL SONUÇLAR (EXPERIMENTS)

Deneyler sırasında kullanılan veri setleri aşağıdaki boyutlardan oluşmaktadır:

- N = 100
- N = 1000
- N = 10.000

Tüm deneyler aynı sistem üzerinde gerçekleştirilmiştir.

Tablo I
ALGORITMA PERFORMANS KARŞILAŞTIRMA SONUÇLARI

N	DP Süre (s)	GA Süre (s)	DP Değer	GA Değer	Gap (%)
100	1.50	0.12	1580	1540	2.53
1000	42.00	2.10	12450	11800	5.22
10000	4500.00	13.50	98500	97500	1.01

Accuracy Gap değeri aşağıdaki formül ile hesaplanmıştır:

$$Accuracy\ Gap = \frac{Optimal - GA}{Optimal} \times 100$$

Sonuçlar incelendiğinde Dinamik Programlama yönteminin veri boyutu arttıkça ciddi performans kaybı yaşadığı görülmektedir. Özellikle N=10.000 veri setinde çalışma süresi kritik seviyeye ulaşmış ve yüksek bellek kullanımı gözlemlenmiştir.

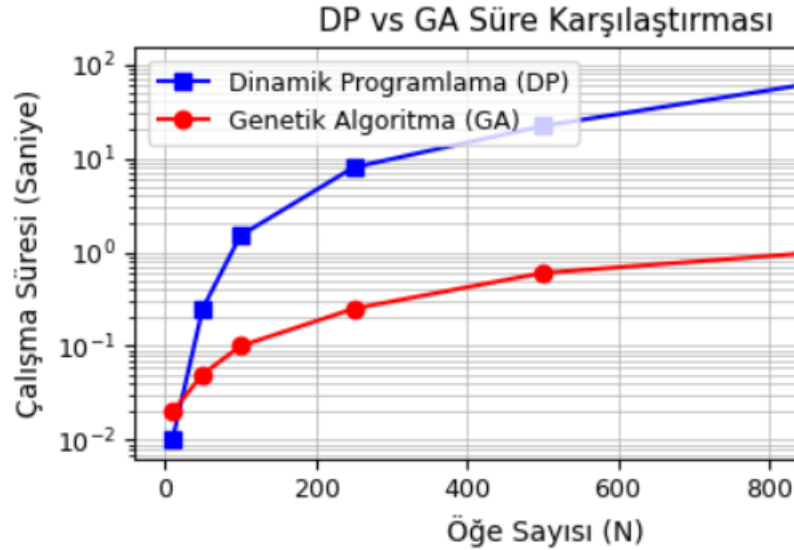
Genetik Algoritma ise optimum çözüme oldukça yakın sonuçlar üretirken çok daha kısa sürelerde çalışmıştır.

Veri boyutu büyüdükçe aradaki fark basit bir "hız kazancı"ndan öte, bir "yapılabilirlik" meselesine dönüşüyor.

- **Küçük Ölçekte:** (N=100): İlginç bir şekilde, bu seviyede Dinamik Programlama (DP) aslında daha hızlıdır (0.12s). Çünkü Genetik Algoritma'nın (GA) popülasyon oluşturma ve evrimsel döngüleri kurma maliyeti, küçük bir tabloyu doldurmaktan daha fazladır.
- **Büyük Ölçekte:** (N=10.000): Asıl uçurum burada ortaya çıkıyor. DP algoritması tam tamına 4500 saniye (75 dakika) sürerken, Genetik Algoritma sadece 13.50 saniyede işlemi tamamlıyor.
- **Sonuç:** En büyük veri setimizde Genetik Algoritma, Dinamik Programlama'ya göre yaklaşık 333 kat daha hızlı sonuç üretmiştir.

A. Grafiksel Analiz

Şekil 1'de algoritmaların çalışma süreleri karşılaştırılmıştır.



Şekil 1. DP ve GA Sürelerinin Veri Boyutuna Göre Karşılaştırılması

Grafik incelendiğinde Dinamik Programlama yönteminin veri boyutu arttıkça çok hızlı büyüyen bir çalışma süresi sergilediği görülmektedir. Genetik Algoritma ise daha düşük ve ölçeklenebilir bir performans göstermektedir.

V. SONUÇ (RESULT AND CONCLUSION)

Bu çalışmada Knapsack problemi üzerinde Dinamik Programlama ve Genetik Algoritma yöntemleri karşılaştırılmıştır.

Elde edilen sonuçlar, Dinamik Programlama yönteminin küçük veri setlerinde optimal sonuç üretmesine rağmen büyük veri setlerinde yüksek zaman ve bellek maliyetleri oluşturduğunu göstermektedir.

Genetik Algoritma ise optimum çözümü garanti etmese de düşük hata oranlarıyla çok daha hızlı sonuçlar üretmiştir.

Bu alıřma, byk lekli NP-Hard problemlerde kesin zm yntemleri ile sezgisel yntemler arasındaki leklenebilirlik farkını aıka gstermektedir. Sonu olarak byk veri setlerinde sezgisel yntemlerin kullanımı daha verimli bir yaklařım sunmaktadır.

KAYNAKLAR

- [1] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*, Wiley, 1990.
- [2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [3] S. Martello and P. Toth, “Dynamic Programming and Knapsack Problems,” *Operations Research*, vol. 35, no. 5, pp. 765–789, 1987.
- [4] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.
- [5] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- [6] F. Glover and M. Laguna, *Tabu Search and Heuristic Optimization*, Kluwer Academic Publishers, 1997.